

**LAPORAN PRAKTIKUM
Teknik Kompilasi**

Dosen Pengampu :

AGUNG PERDANANTO S.Kom,M.Kom



Disusun oleh:

Nama : Septian Handita Surya

NIM : 231011400174

Kelas : 06TPLM002

**TEKNIK INFORMATIKA
FAKULTAS ILMU
KOMPUTER UNIVERSITAS
PAMULANG TAHUN
2025/2026**

BAB 1

Pendahuluan

1.1 Latar Belakang

Kompiler merupakan perangkat lunak yang berfungsi menerjemahkan bahasa tingkat tinggi menjadi bahasa mesin melalui beberapa tahapan, yaitu analisis leksikal, sintaksis, semantik, dan generasi kode.

Pada tugas ini dilakukan simulasi proses kompilasi menggunakan konstruksi perulangan **while** untuk memahami bagaimana suatu program diproses hingga menghasilkan Three Address Code (TAC).

1.2 Tujuan

Tujuan dari proyek ini adalah:

- Memahami tahapan kompilasi.
- Mengimplementasikan analisis leksikal.
- Membentuk Abstract Syntax Tree (AST).
- Melakukan pengecekan semantik.
- Menghasilkan Three Address Code (TAC).

BAB II

Dasar Teori

2.1 Analisis Leksikal

Analisis leksikal bertugas memecah source code menjadi token-token yang dikenali compiler.

Contoh:

```
while (x < 10)
```

```
{
```

```
    x = x + 1;
```

```
}
```

Token :

WHILE

(

ID(x)

<

NUM(10)

)

```
{
```

ID(x)

=

ID(x)

+

NUM(1)

```
}
```

2.2 Analisis Sintaksis

Analisis sintaksis memeriksa apakah urutan token sesuai dengan aturan grammar.

BNF:

$\langle \text{while_stmt} \rangle ::= \text{"while" "(" } \langle \text{condition} \rangle \text{ ")" "{" } \langle \text{statement} \rangle \text{ "}"}$

$\langle \text{condition} \rangle ::= \langle \text{identifier} \rangle \langle \text{operator} \rangle \langle \text{value} \rangle$

$\langle \text{statement} \rangle ::= \langle \text{identifier} \rangle \text{"=" } \langle \text{expression} \rangle$

$\langle \text{expression} \rangle ::= \langle \text{identifier} \rangle \text{"+" } \langle \text{value} \rangle$

2.3 Analisis Semantik

Tahap ini memvalidasi:

- keberadaan variabel
- tipe data
- kesesuaian operasi

Contoh:

$x < 10$

Valid.

Sedangkan:

$\text{nama} < 10$

Tidak Valid:

2.4 Three Address Code

TAC merupakan representasi kode antara yang digunakan compiler sebelum menghasilkan kode mesin.

Contoh:

L1:

ifFalse $x < 10$ goto L2

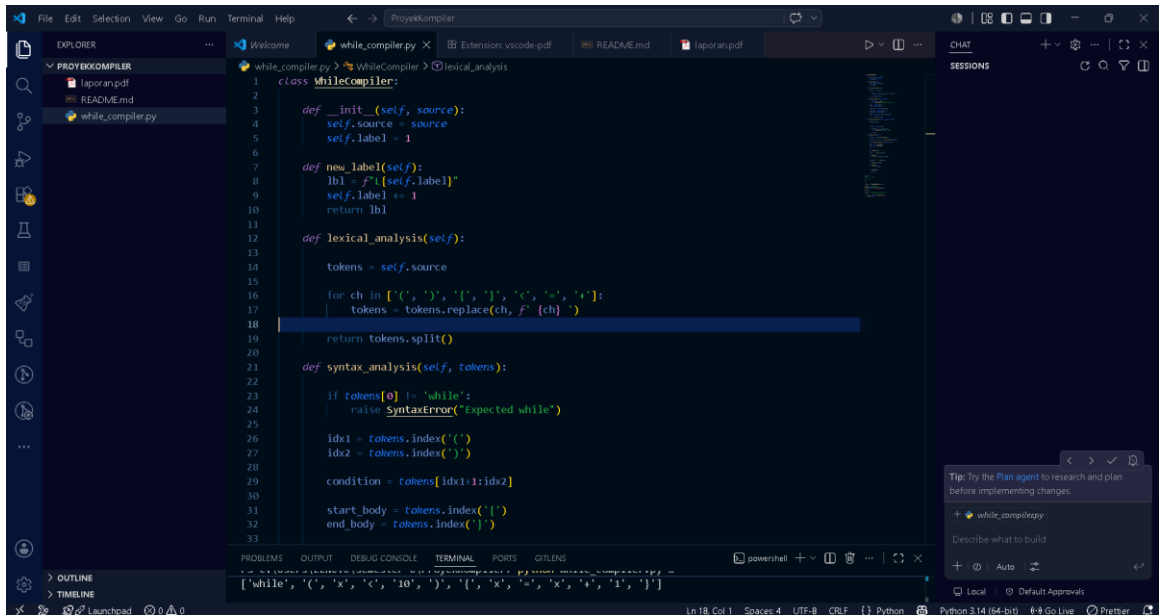
$x = x + 1$

goto L1

L2:

BAB III

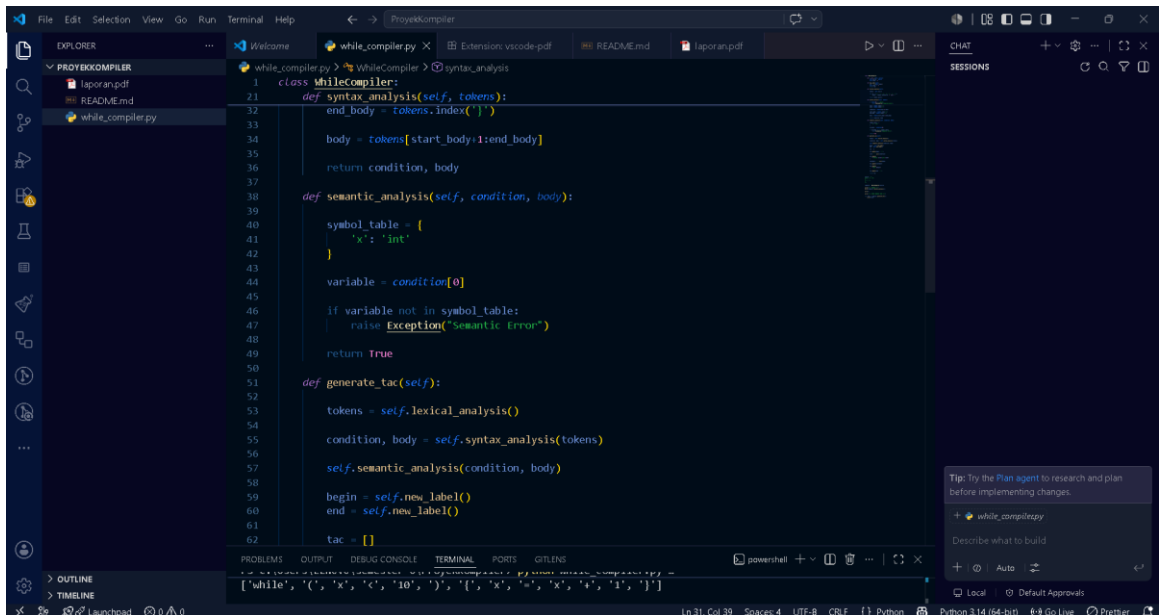
IMPLEMENTASI



```
1 class WhileCompiler:
2
3     def __init__(self, source):
4         self.source = source
5         self.label = 1
6
7     def new_label(self):
8         lbl = f'({self.label})'
9         self.label += 1
10        return lbl
11
12    def lexical_analysis(self):
13        tokens = self.source
14
15        for ch in ['(', ')', '[', ']', '{', '}', '=', '+', '*']:
16            tokens = tokens.replace(ch, f' {ch} ')
17
18        return tokens.split()
19
20    def syntax_analysis(self, tokens):
21        if tokens[0] != 'while':
22            raise SyntaxError("Expected while")
23
24        idx1 = tokens.index('(')
25        idx2 = tokens.index(')')
26
27        condition = tokens[idx1+1:idx2]
28
29        start_body = tokens.index('{')
30        end_body = tokens.index('}')
31
32        tokens = tokens[:start_body] + tokens[end_body:]
33
```

Terminal output:

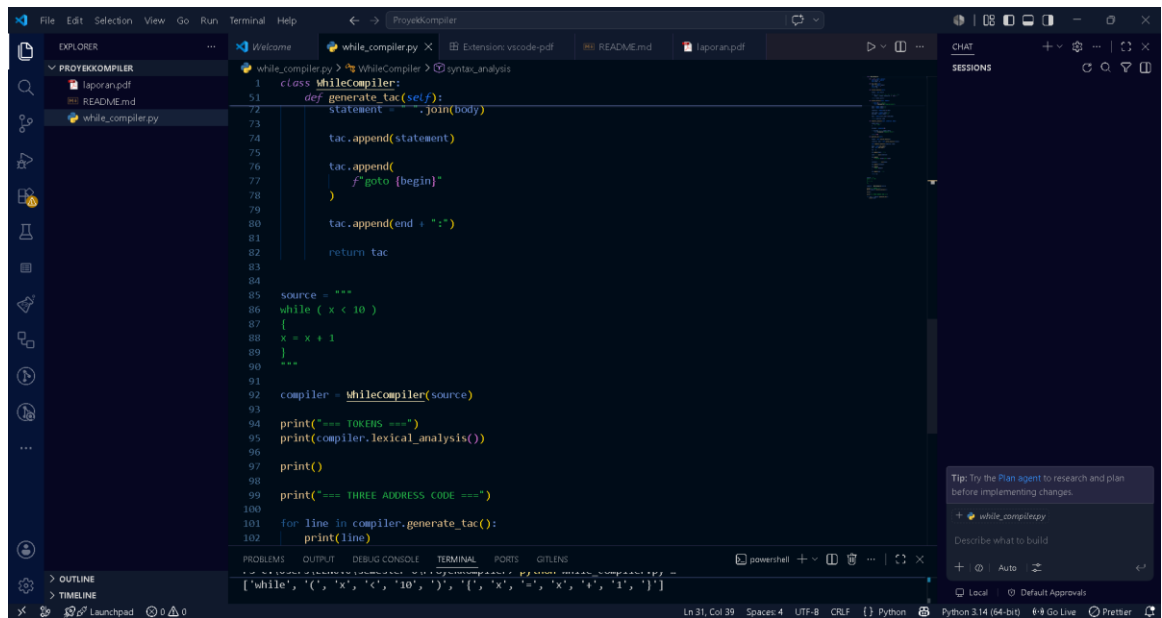
```
[ 'while', '(', 'x', '=', '10', ')', '{', 'x', '=', 'x', '+', '1', '}', ']' ]
```



```
21    def syntax_analysis(self, tokens):
22        end_body = tokens.index('}')
23
24        body = tokens[start_body+1:end_body]
25
26        return condition, body
27
28    def semantic_analysis(self, condition, body):
29
30        symbol_table = {
31            'x': 'int'
32        }
33
34        variable = condition[0]
35
36        if variable not in symbol_table:
37            raise Exception("Semantic Error")
38
39        return True
40
41    def generate_tac(self):
42        tokens = self.lexical_analysis()
43
44        condition, body = self.syntax_analysis(tokens)
45
46        self.semantic_analysis(condition, body)
47
48        begin = self.new_label()
49        end = self.new_label()
50
51        tac = []
52
```

Terminal output:

```
[ 'while', '(', 'x', '=', '10', ')', '{', 'x', '=', 'x', '+', '1', '}', ']' ]
```



Input Program :

while (x < 10)

{

 x = x + 1

}

Hasil Analisis Leksikal:

['while',

('(',

'x',

'<',

'10',

'),

{',

'x',

'=',

'x',

+',

'1',

']']

Hasil Analisis Semantik:

Variabel x ditemukan

Tipe data int

Semantik valid

Hasil TAC :

L1:

ifFalse x < 10 goto L2

x = x + 1

goto L1

L2:

BAB IV

KESIMPULAN

Berdasarkan hasil implementasi, dapat disimpulkan bahwa proses kompilasi terdiri dari beberapa tahapan utama yaitu analisis leksikal, sintaksis, semantik, serta generasi kode antara.

Implementasi konstruksi perulangan while berhasil menghasilkan token, AST, validasi semantik, dan Three Address Code sebagai representasi kode antara.

DAFTAR PUSTAKA

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006).

Compilers: Principles, Techniques, and Tools (2nd Edition).

Pearson.

Louden, K. C. (2003).

Compiler Construction: Principles and Practice.

Thomson Learning.

Python Software Foundation.

Python Documentation.

<https://docs.python.org/>

IEEE Computer Society.

Compiler Design Fundamentals.